Embedded Controller

Interrupt Exception

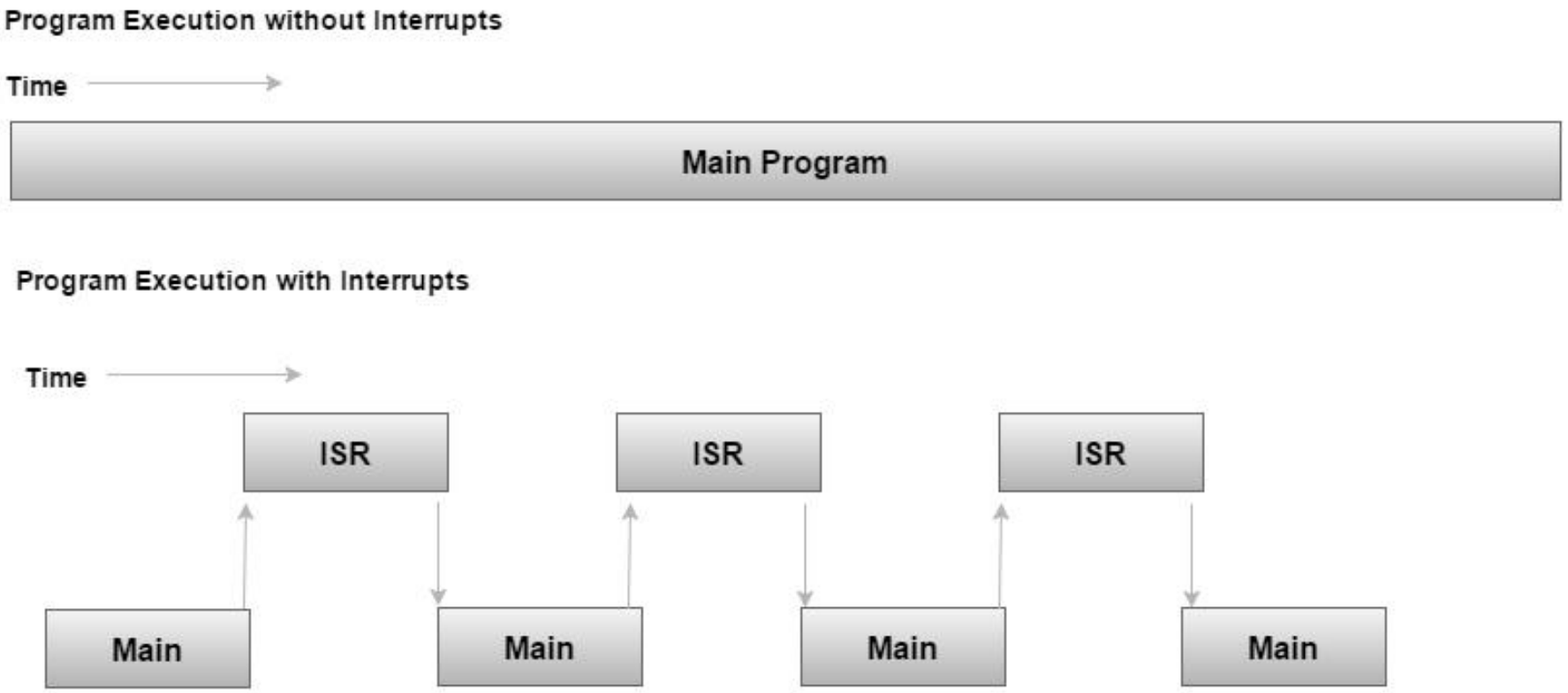
Young-Keun Kim
School of Mechanical & Control Engineering

Handong Global University

Last modified: 2025.8

Interrupts

- **What is an Interrupt?**
  - Inform a program of some external events at instance or timely
    - Polling vs Interrupt
  - Implement multi-tasking with priority support



ISR : Interrupt Service Routine

[tutorialpoints]

한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **Examples**
  - Turn On LED immediately at the instance a button is pressed
    → while the processor is doing other tasks
  - Temporarily pause the current task to read the incoming message
  - Calling a process at a regular period

Polling *vs* Interrupt

```
// Polling method
while (1) {
    read_button_input;
    if (pushed)
        exit;
}
turn_on_LED;
```

vs

```
// Interrupt method
while (1) {
}
…

interrupt_handler(){
 turn_on_LED;
 exit;
}
```


한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **Exception**
  - Exception
    - Interrupt
    - Reset, hardware/software faults
    - Memory violation, debug events

  - Synchronous Exception: unexpected faults
    - debugging, Zero_DIV error etc

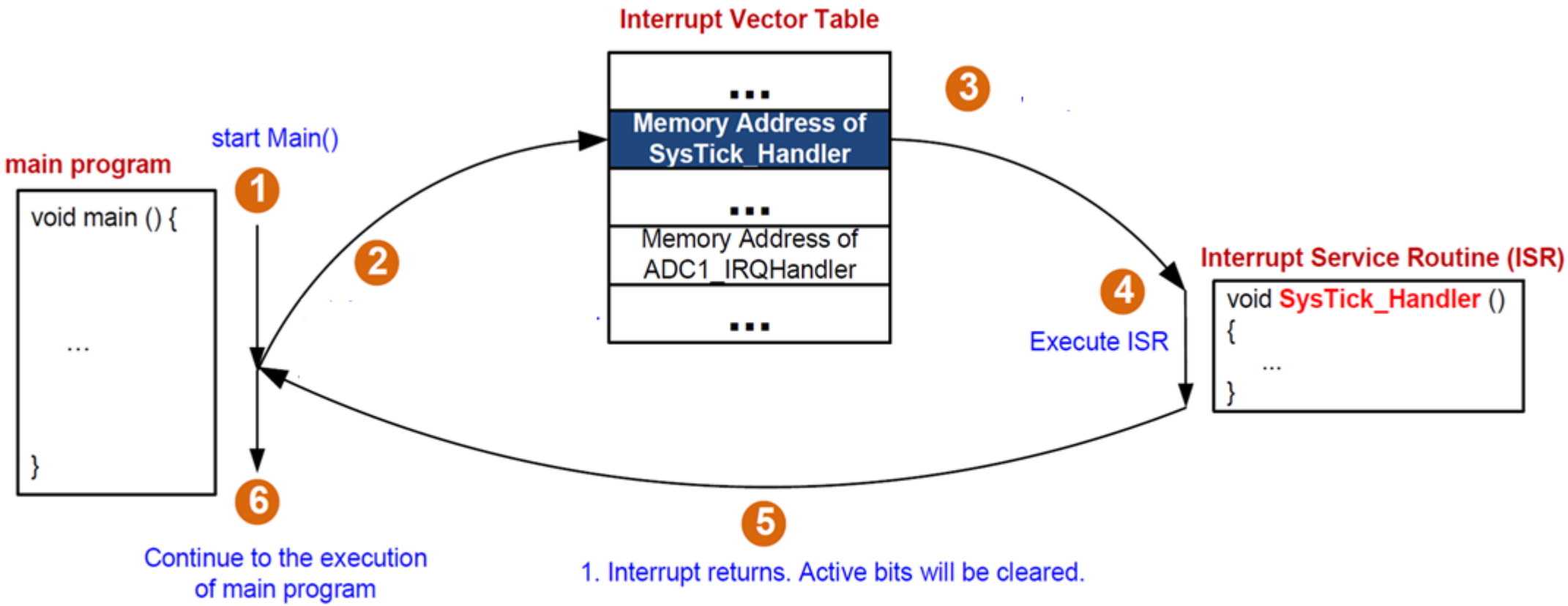
  - **Asynchronous Exception**: exception with a purpose → Interrupts
    - System Timer (SysTick)
    - External Interrupt (EXTI)
    - Software Interrupt (SVC), ..

한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **What happens at interrupt? overview**

  eg) SysTick Interrupt



한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

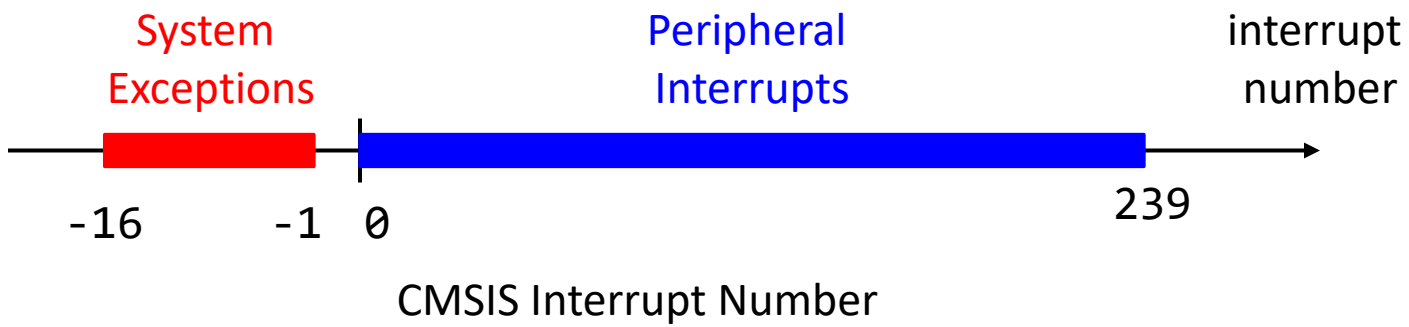
- **Terminology**
  - **IRQn**: Interrupt ReQuest number
  - **Priority:** lower value has a higher priority
  - **ISR:** Interrupt Service Routine
    - Predefined program function for interrupt
    - Function name is usually defined as `void InterruptName_handler()`
  - **Exception Vector Table** (ISR Vector Table)
    - Stores address of each ISR
  - **NVIC**: Nested Vectored Interrupt Controller
    - Nested: can call other interrupt of higher priority during an interrupt
    - Vectored: can directly go to the ISR address and execute

한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **IRQn**
  - For Cortex-M4, IRQ numbers (IRQn) are
    - Number of Interrupts: max 256
    - **System** Exception
      : IRQn # -16 ~ -1
    - **Peripheral** Interrupt
      : IRQ # 0 ~ Pos. number

Exception number[1]	IRQ number[1]	Exception type	Priority	Vector address or offset[2]	Activation
15	-1	SysTick	Configurable[3]	0x0000003C	Asynchronous
16 and above	0 and above	Interrupt (IRQ)	Configurable[4]	0x00000040 and above[5]	Asynchronous



CMSIS Interrupt Number

- In CMSIS programming, we do not use negative numbers.
- For IRQn (-16~ 240) → CMSIS Exception number = 16 + n

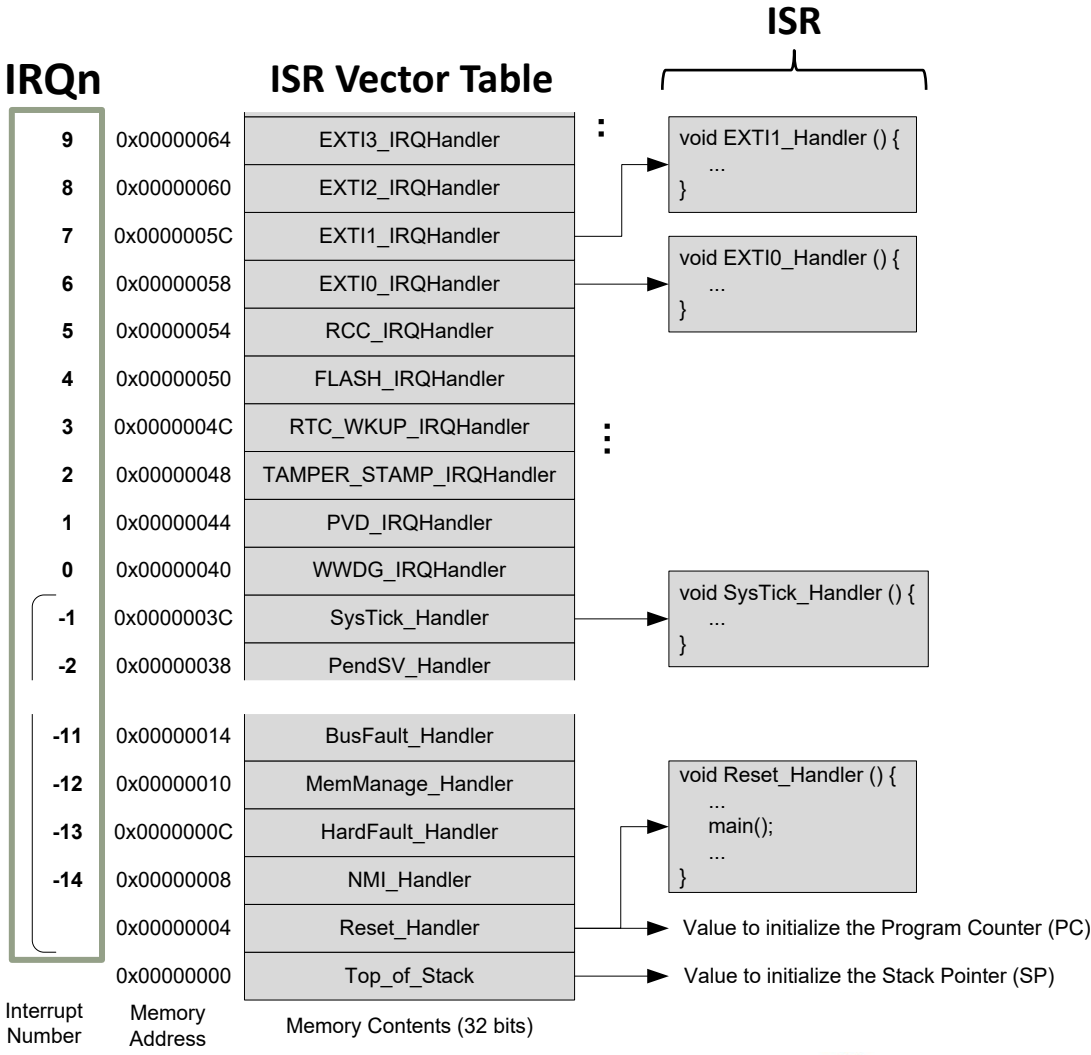
한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **ISR**
  - Which 'program function' does it call when an interrupt occurs?
    - ISR: Interrupt Service Routine
    - Every interrupt has predefined ISR
    - Defined as `void InterruptName_IRQHandler()`

      SysTick Interrupt → void SysTick_Handler() {..}

  - Where is ISR defined in memory?
    - **ISR Vector Table**
    - Address book for ISR



한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **How is interrupt managed?**
  - **at system reset:**
    - All interrupts are disabled
    - All interrupts have priority level of 0 (highest programmable level)
    - All interrupt pending status are cleared

  - **at interrupt active**
    - Cannot accept the same interrupt request again
    - It must be completed and terminated with an exception return

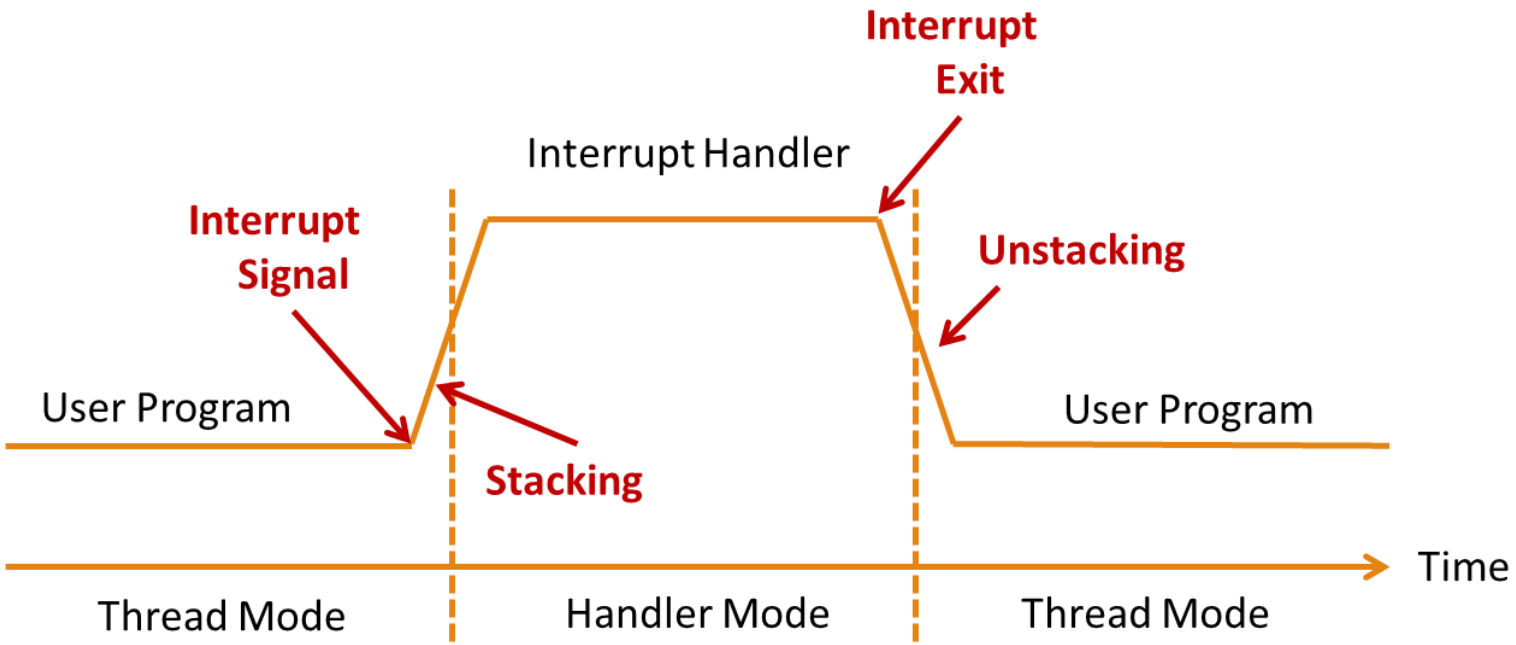
한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **How is interrupt managed?**

When Interrupt is called,

- Stacks current program counter: to preserve the running environment
- Then, calls Interrupt_Handler(ISR) to execute
- Returns and unstacks program counter to recover the environment

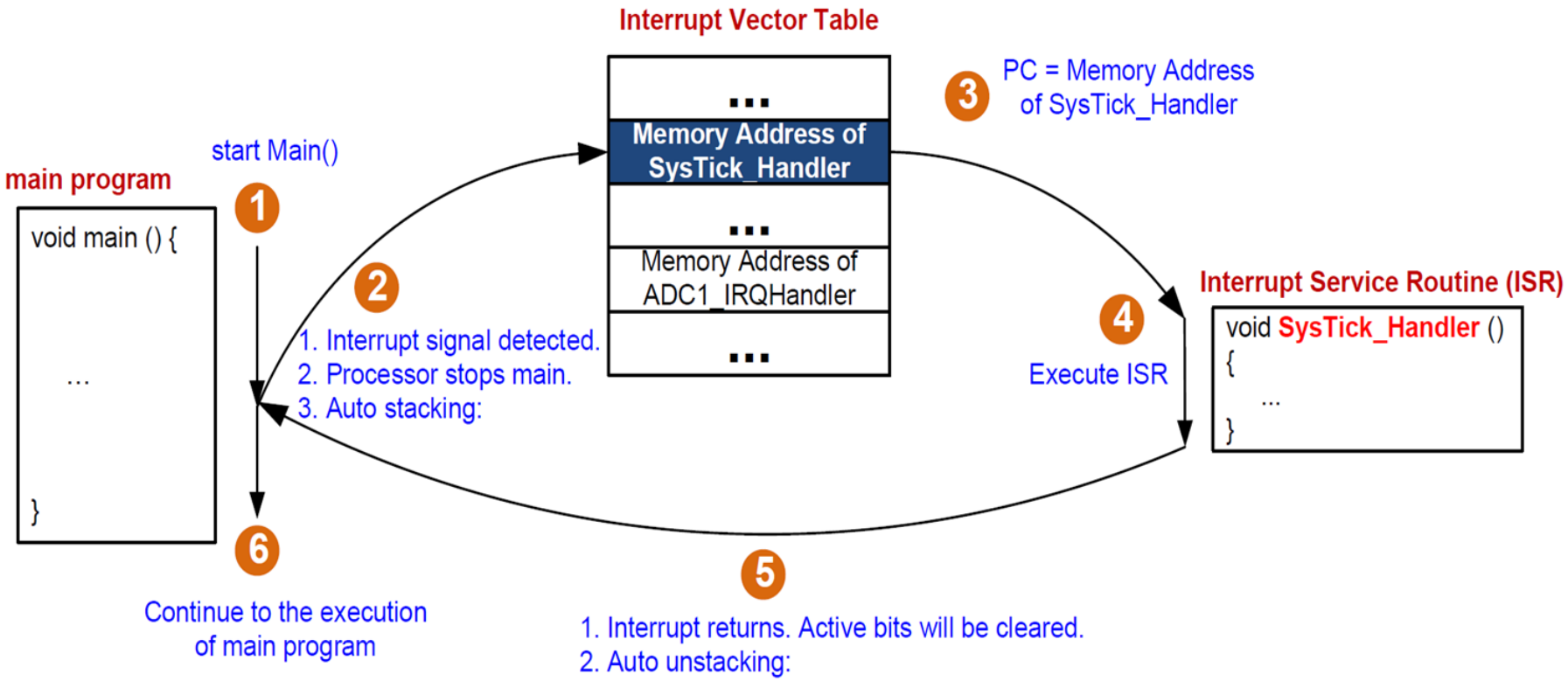


한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **How is interrupt managed?**

Example: SysTick Interrupt



한동대학교 HANDONG GLOBAL UNIVERSITY

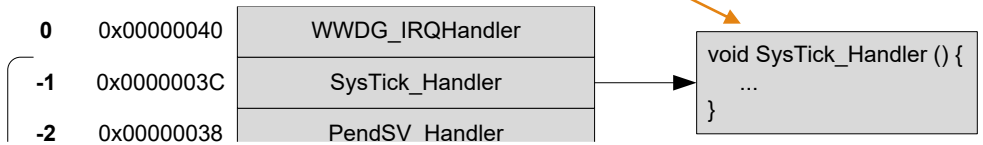
Interrupts

- **Who defined the ISR handler name?**
  - Example: **SysTick_Handler()**

  - It is defined at " startup code: **startup_stm32f4xxx.s**"

```
__Vectors       DCD     __initial_sp              ; Top of Stack
                DCD     Reset_Handler             ; Reset Handler
                DCD     NMI_Handler               ; NMI Handler
                DCD     HardFault_Handler         ; Hard Fault Handler
                DCD     MemManage_Handler         ; MPU Fault Handler
                DCD     BusFault_Handler          ; Bus Fault Handler
                DCD     UsageFault_Handler        ; Usage Fault Handler
                DCD     SVC_Handler               ; SVCall Handler
                DCD     DebugMon_Handler          ; Debug Monitor Handler
                DCD     0                         ; Reserved
                DCD     PendSV_Handler            ; PendSV Handler
                DCD     SysTick_Handler           ; SysTick Handler
```

```
SysTick_Handler PROC
                EXPORT  SysTick_Handler           [WEAK]
                B       .
                ENDP
```



한동대학교 HANDONG GLOBAL UNIVERSITY

Interrupts

- **Who defined the ISR handler name?**
  - Example: **Reset_Handler**
    - When the processor is reset and powered up → calls " Reset_Handler"
    - In Reset_handler, it calls **main()**

```
; Reset handler
Reset_Handler   PROC
                EXPORT  Reset_Handler             [WEAK]
        IMPORT  SystemInit
        IMPORT  __main

                LDR     R0, =SystemInit
                BLX     R0
                LDR     R0, =__main
                BX      R0
                ENDP
```


한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC

- **How does MCU manage ISR functions?**
  - uses NVIC (Nested Vectored Interrupt Controller)

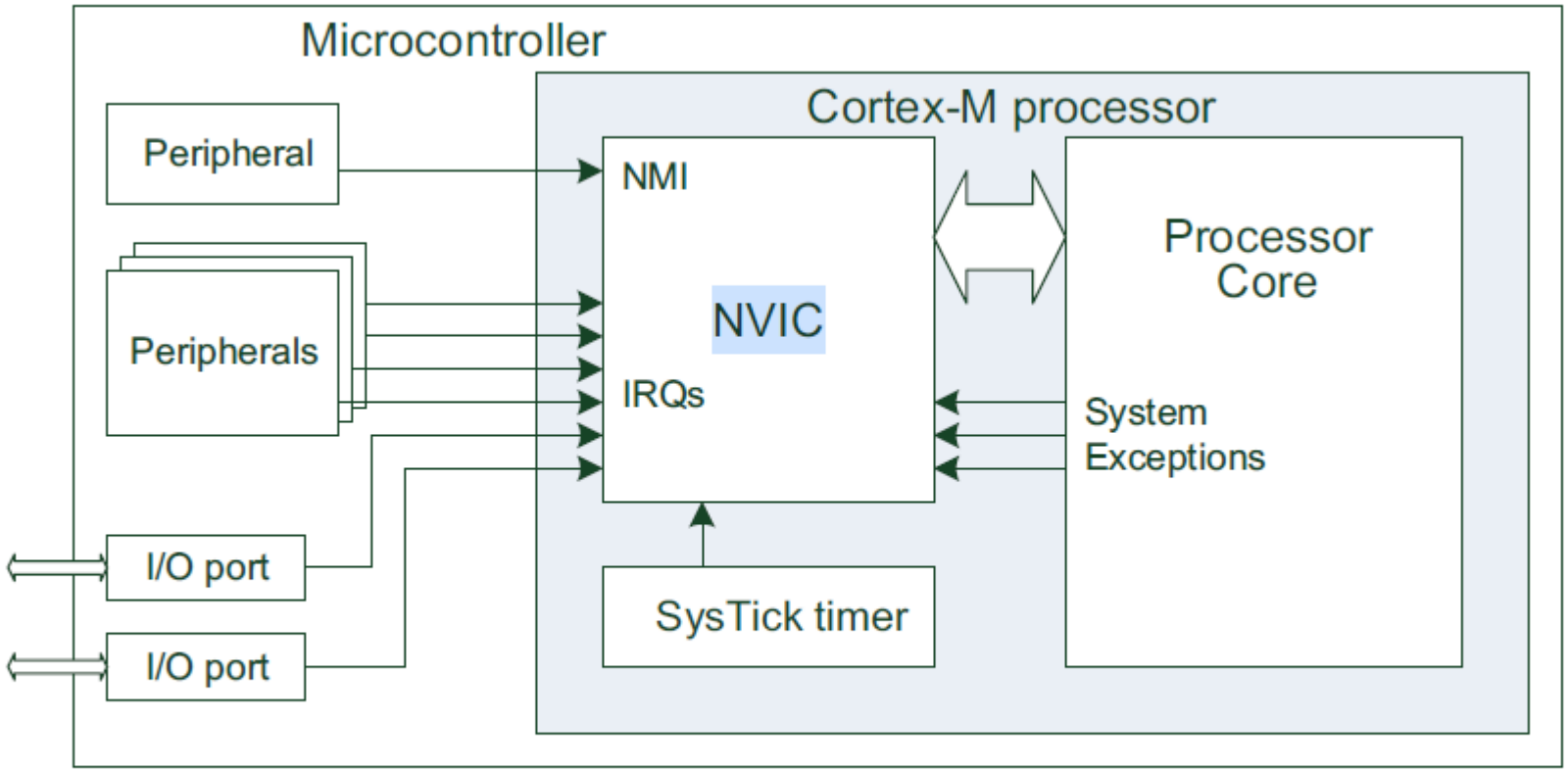


FIGURE 7.1

Various sources of exceptions in a typical microcontroller

한동대학교 HANDONG GLOBAL UNIVERSITY

NVIC

- **NVIC** is used to manage interrupts
  - Enable, Disable, Set Priority level
  - Arm Cortex M4: 52 maskable interrupt channel, 16 priority levels

- **NVIC (Nested Vectored Interrupt Controller)**
  - Interrupt set-enable register: NVIC_ISERx
  - Interrupt clear-enable register: NVIC_ICERx
  - Interrupt set-pending register: NVIC_ISPRx
  - Interrupt clear-pending register: NVIC_ICPRx
  - Interrupt active bit register: NVIC_IABRx
  - Interrupt priority register: NVIC_IPRx
  - Software trigger interrupt register: NVIC_STIR

[Programming Manual: Ch 4.3]

Use CMSIS functions:

```
// Disable interrupt
NVIC_DisableIRQ (IRQn);

// Enable interrupt
NVIC_EnableIRQ (IRQn);

// clear pending status
NVIC_ClearingPending (IRQn);

// set priority level
NVIC_SetPriority (IRQn, priority);
```

read [EC] NVIC_Register.pdf

한동대학교 HANDONG GLOBAL UNIVERSITY

- **Programming example**
  - Setting Interrupt Priority of EXTI_0

Use CMSIS functions:

```
// Disable interrupt
NVIC_DisableIRQ (IRQn);

// Enable interrupt
NVIC_EnableIRQ (IRQn);

// clear pending status
NVIC_ClearingPending (IRQn);

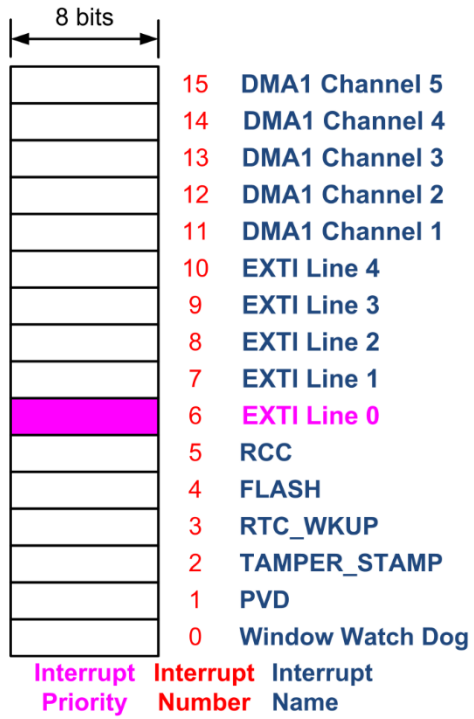
// set priority level
NVIC_SetPriority (IRQn, priority);
```

Example Code:

```
/**  main.c

# include <core_cm4.h>
void main()
{        …
       // Set IRQn=6 (EXTI_0) as priority of 15
       // NVIC_SetPriority (IRQn, priority);

       NVIC_SetPriority(6, 15);
       …
}
```

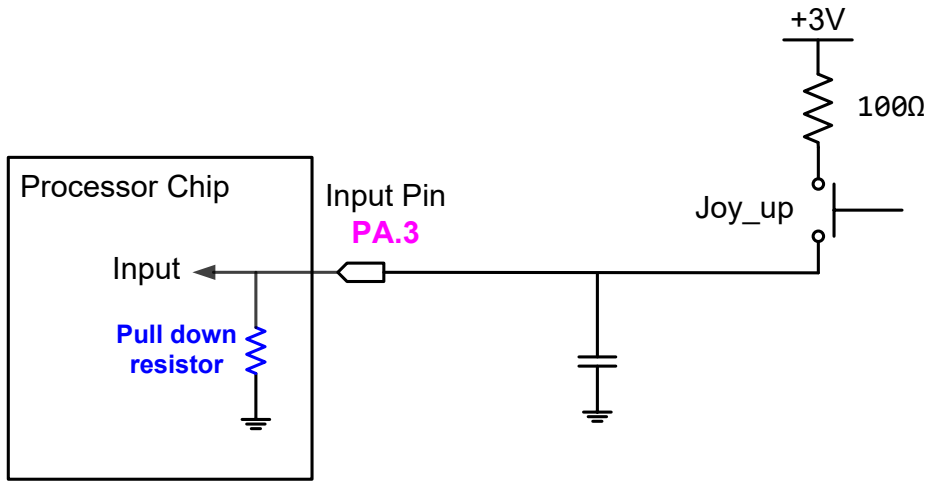


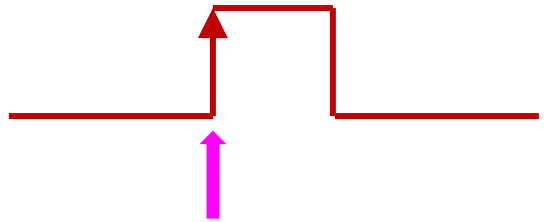
Interrupt Case Study : External Interrupt

External Interrupt

- **External Interrupt**
  - EXTI0 to EXTI15 : total 16
  - For interrupts triggered by input peripherals
    - Switches, buttons, external pulses and more

  - Each EXTI**x** is matched to pin_**x** of a GPIO port
    - EXTI**0** is matched to only one pin from PA**0**, PB**0**.. or PG**0**
    - EXTI**2** is matched to only one pin from PA**2**, PB**2**.. or PG**2**



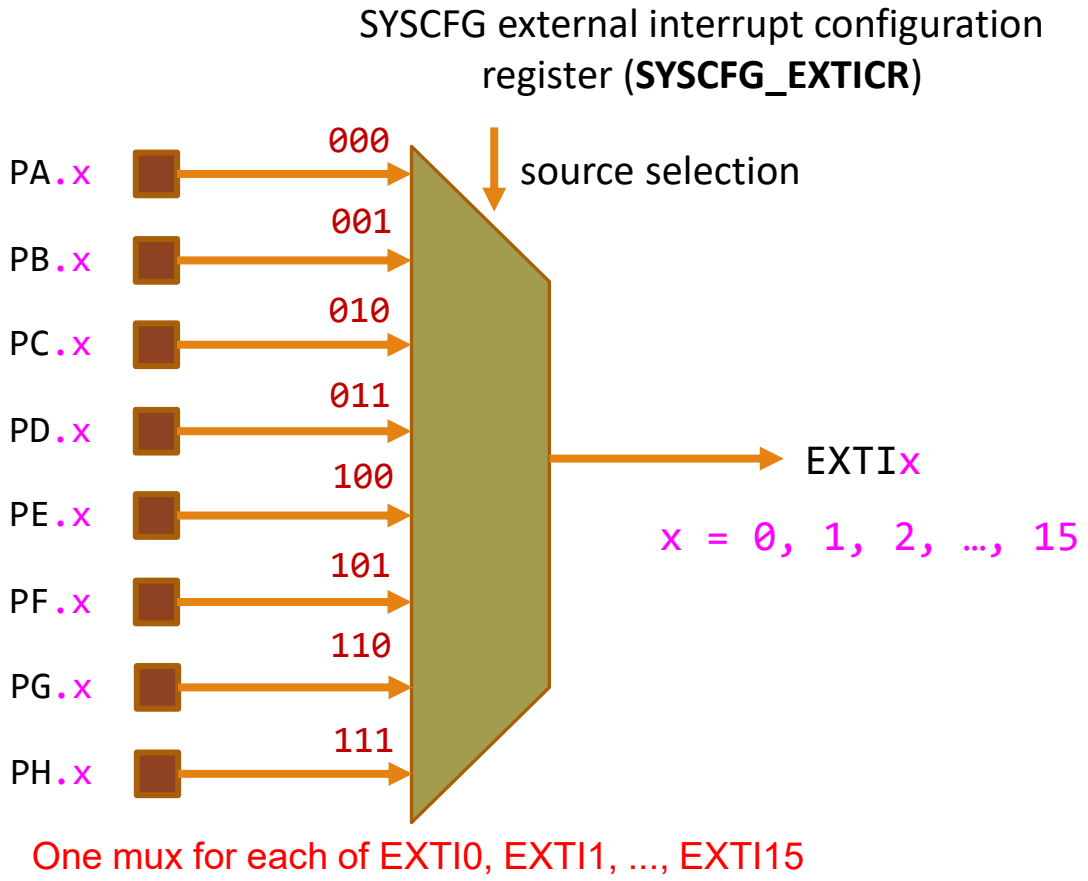


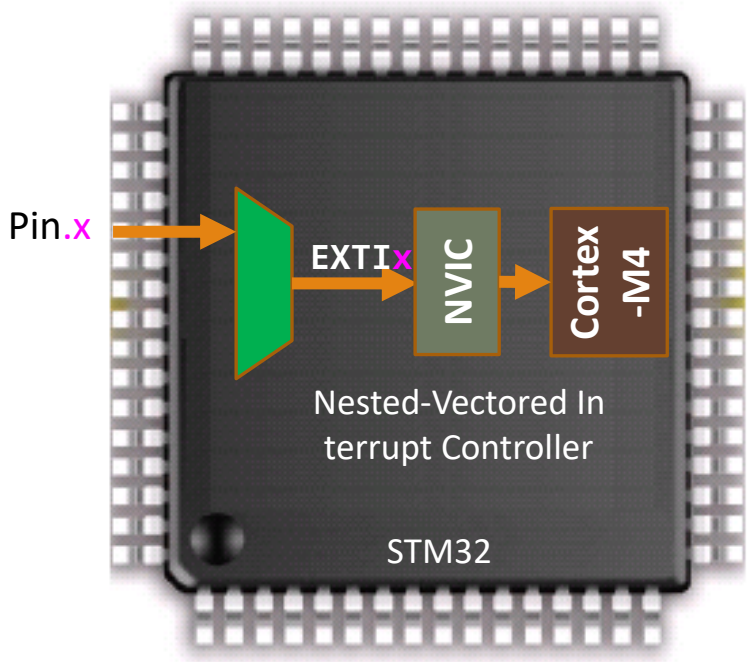
Rising edge triggers EXTI interrupt.

한동대학교 HANDONG GLOBAL UNIVERSITY

External Interrupt

- **MUX for EXTI*x* select**
  - How to select the Port for EXTIx?
  - One MUX for each pin_number → Total of 16 MUXs (EXTI0 ~ EXTI15)



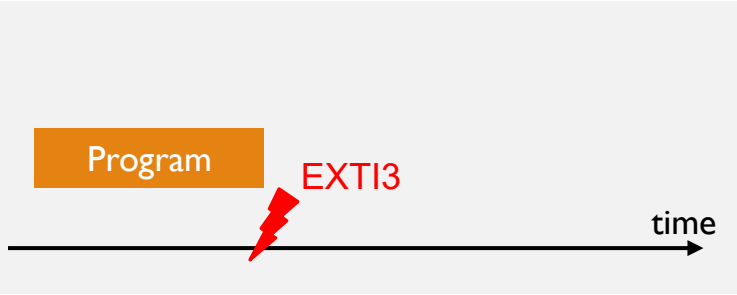


Single Interrupt

Example: External Interrupt

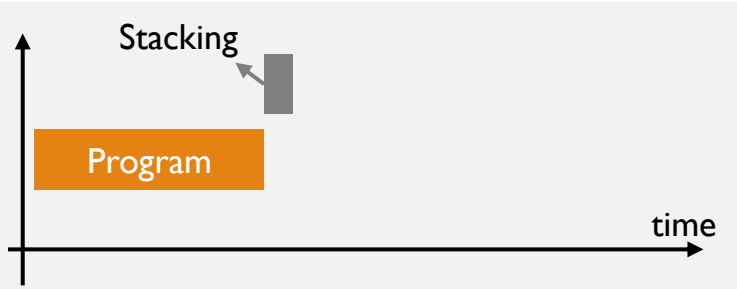
void EXTI3_IRQHandler() // IRQn=9

NVIC enables EXTI3



	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

NVIC Sets Pending Bit for EXTI3



	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

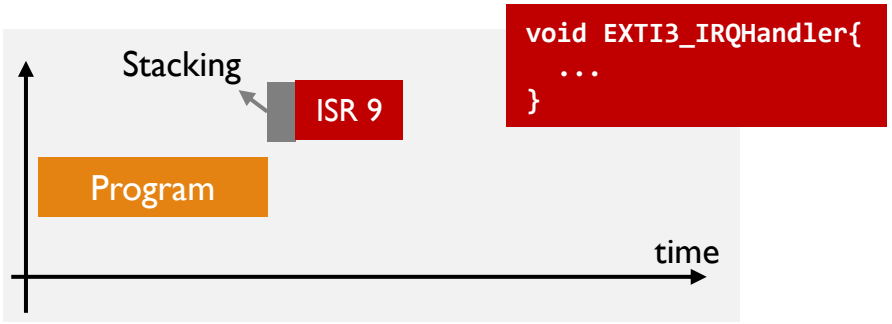
한동대학교 HANDONG GLOBAL UNIVERSITY

Single Interrupt

Example: External Interrupt

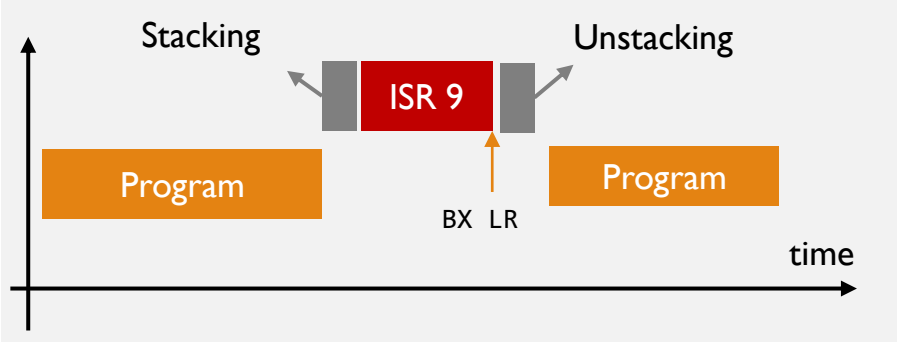
void EXTI3_IRQHandler() // IRQn=9

NVIC changes from **Pending → Active** state.



	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

NVIC clears Active Register → 0



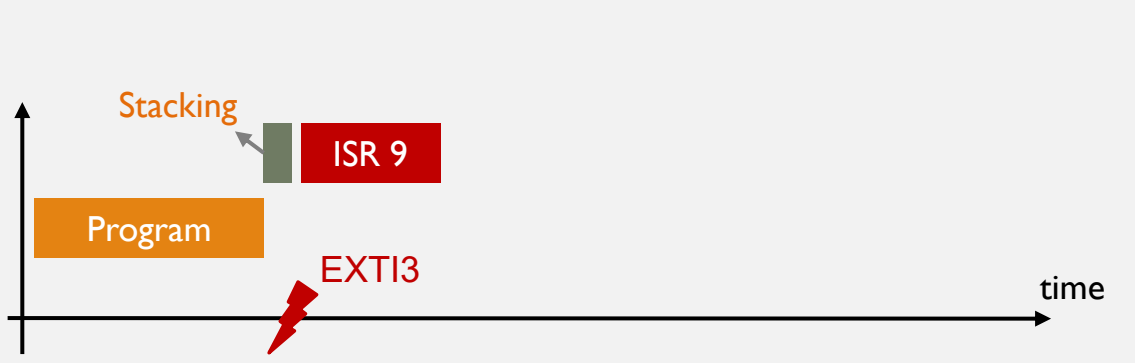
	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

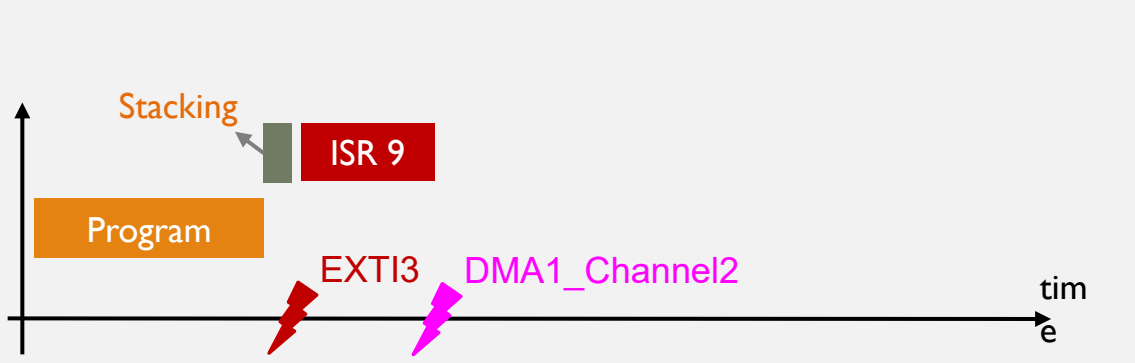
Nested Interrupts: Example of Preemption

When multiple ISRs are called :

Example: DMA1 Interrupt are called while in EXTI3

```
void EXTI3_IRQHandler()          // IRQn=9
void DMA1_IRQHandler()           // IRQn=12
```





NVIC responds to higher priority ISR

NVIC first executes EXTI3(IQRn=9)
→ void EXTI3_IRQHandler()

	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3

	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	1	0	0	0	0
Priority Register	3	4	7	5	3

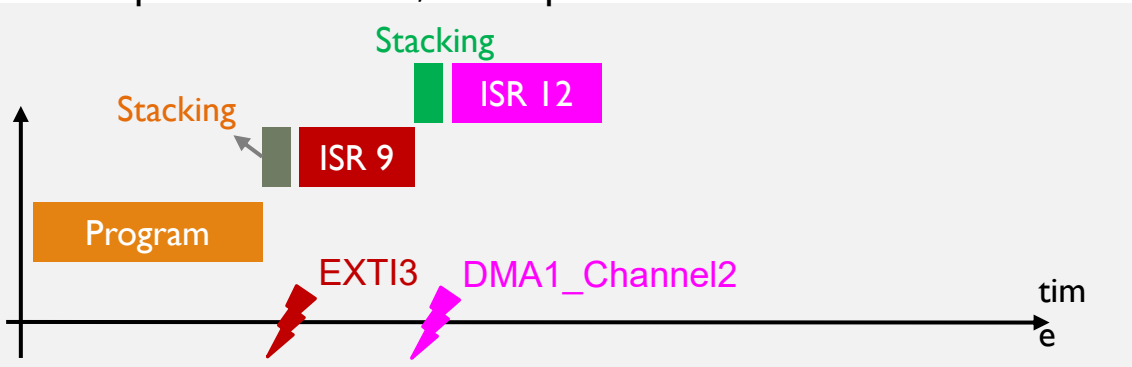
Lower priority value means higher urgency.

한동대학교 HANDONG GLOBAL UNIVERSITY

Nested Interrupts: Example of Preemption

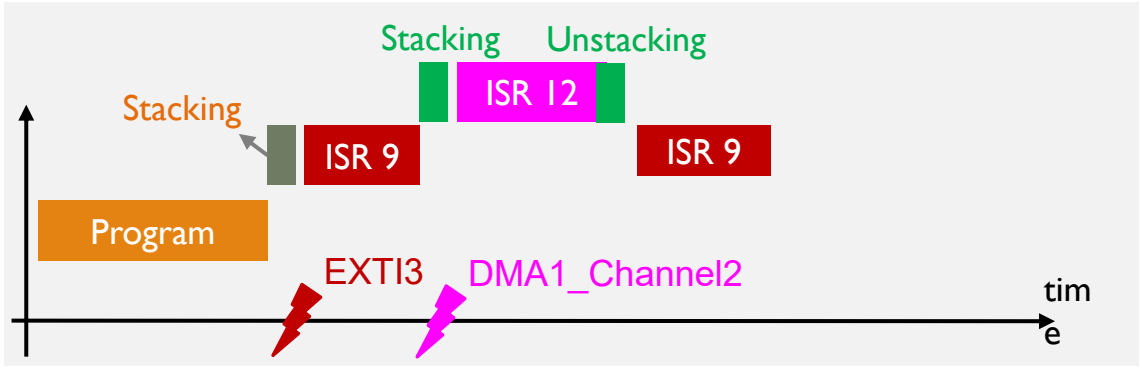
When multiple ISRs are called : DMA1 Interrupt are called while in EXTI3

NVIC pauses EXTI3, and process ISR12



	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	1	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3

EXTI3 is re-activated after DMA1 is completed

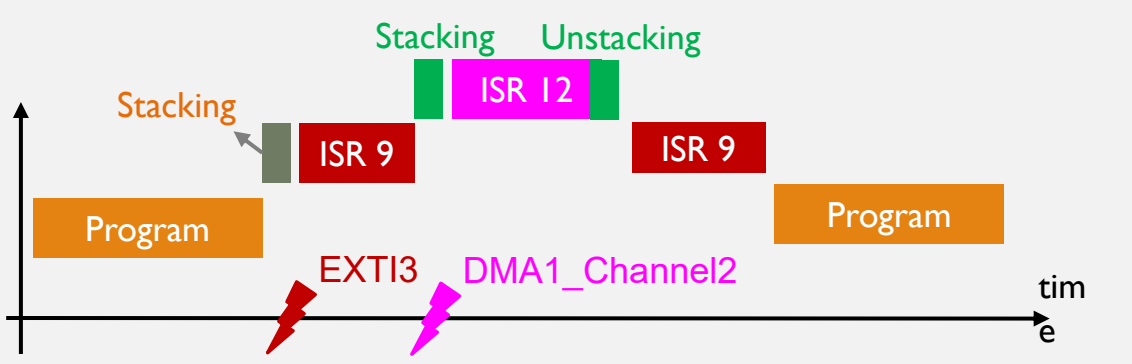


	DMA1_Channel2			EXTI3	
	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3

Nested Interrupts: Example of Preemption

When multiple ISRs are called : DMA1 Interrupt are called while in EXTI3

Returns to the previous environment where it was left off.



	DMA1_Channel2			EXTI3	
Interrupt Number	12	11	10	9	8
Enable Register	1	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	5	3

Interrupt preemption: temporarily stopping the current interrupt handler to start another interrupt

한동대학교 HANDONG GLOBAL UNIVERSITY